204252 Data Structures and Analysis

Chapter 11: Graphs (Part 3) - Minimum Spanning Trees

Lecturer

August 31, 2026

Computer Science Department

Outline

Minimum Spanning Tree (MST) Concepts

Prim's Algorithm

Kruskal's Algorithm

Applications of MST

Minimum Spanning Tree (MST) Concepts

Definition of Spanning Trees

A **Spanning Tree** of an undirected, connected graph $G = (V, E)$ is a subgraph that is a tree (acyclic) and spans all vertices in V. **Properties**:

- A spanning tree exists if and only if G is **connected**.
- It contains exactly $|V|$ vertices and $|V| - 1$ edges.
- Adding any single edge to a spanning tree creates a unique cycle.
- Removing any edge from a spanning tree disconnects the graph.

Minimum Spanning Tree (MST)
For a weighted, undirected graph, the Minimum Spanning Tree is a spanning tree whose sum of edge weights is minimized:

$$w(T) = \sum_{e \in T} w(e)$$

Prim's Algorithm

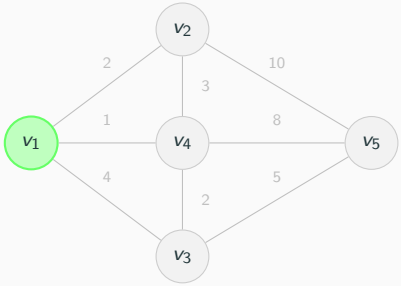
Prim's Algorithm - Overview

- **Greedy Strategy**: Start with a single root vertex, and grow the tree one vertex at a time.
- At each step, choose the cheapest edge that connects a vertex in the tree to a vertex outside the tree.
- Similar to Dijkstra's shortest-path algorithm in structure, but:
  - Dijkstra's: Minimizes total path length from source s ($d_v = d_u + c(u, v)$).
  - Prim's: Minimizes distance to the **growing tree** ($d_v = c(u, v)$).
- **Data Structure**: Priority queue to select the minimum-weight edge efficiently.

Prim's C++ Pseudocode

```cpp
void Graph::Prim(int s) {
    vector<int> dist(numVertices, INT_MAX);
    vector<int> parent(numVertices, -1);
    vector<bool> inMST(numVertices, false);
    dist[s] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true;
        for (auto edge : adjLists[u]) {
            int v = edge.dest, w = edge.weight;
            if (!inMST[v] && w < dist[v]) {
                dist[v] = w; parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Prim's Algorithm Step-by-Step (Initial State (Source = v1))

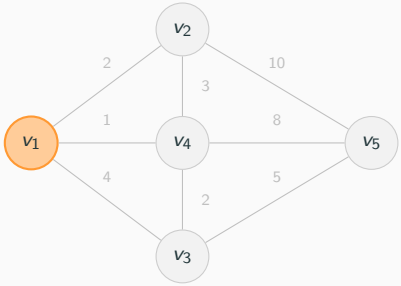


MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	∞	-
v_3	F	∞	-
v_4	F	∞	-
v_5	F	∞	-

Total MST Cost: **0**

Prim's Algorithm Step-by-Step (Step 1: Process v1)

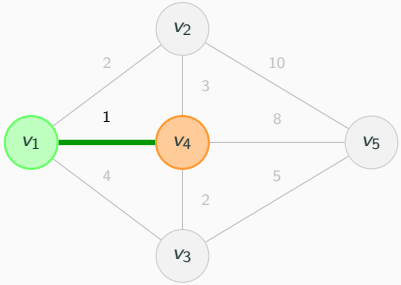


MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	2	v1
v_3	F	4	v1
v_4	F	1	v1
v_5	F	∞	-

Total MST Cost: 1

Prim's Algorithm Step-by-Step (Step 2: Process v4 (Minimum distance to tree))

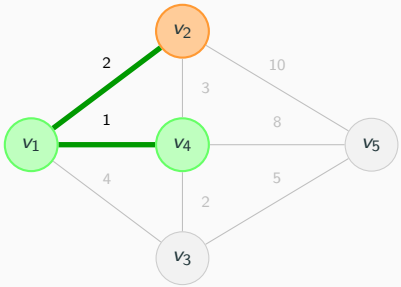


MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	F	2	v1
v_3	F	2	v4
v_4	T	1	v1
v_5	F	8	v4

Total MST Cost: **3**

Prim's Algorithm Step-by-Step (Step 3: Process v2 (Tie-break minimum distance))

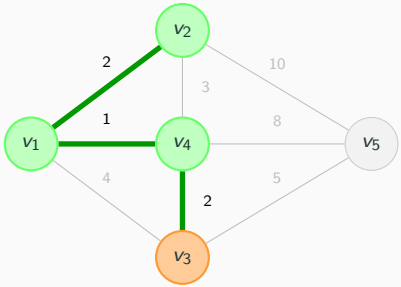


MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	T	2	v1
v_3	F	2	v4
v_4	T	1	v1
v_5	F	8	v4

Total MST Cost: **3**

Prim's Algorithm Step-by-Step (Step 4: Process v3)

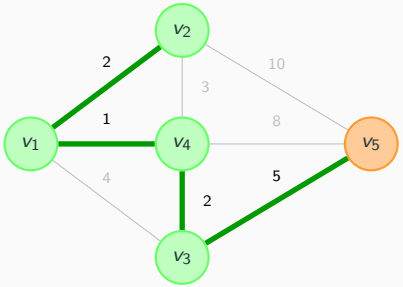


MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	T	2	v1
v_3	T	2	v4
v_4	T	1	v1
v_5	F	5	v3

Total MST Cost: **5**

Prim's Algorithm Step-by-Step (Step 5: Process v5)



MST Construction Table:

Node	Known	Dist d_v	Parent p_v
v_1	T	0	-
v_2	T	2	v1
v_3	T	2	v4
v_4	T	1	v1
v_5	T	5	v3

Total MST Cost: **10**

Kruskal's Algorithm

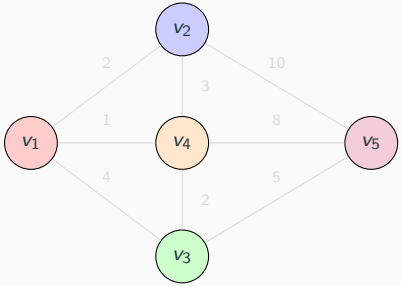
Kruskal's Algorithm - Overview

- **Greedy Strategy**: Maintain a forest of spanning trees. Start with $|V|$ trees (each vertex is a separate tree/set).
- Sort all edges in non-decreasing order of weight.
- Process edges one by one:
  - If the edge connects two vertices in different trees: add it to the MST, and merge the two trees.
  - If they are already in the same tree: discard the edge (adding it would create a cycle).
- **Data Structure**: Use **Disjoint Set Union (DSU)** to perform cycle detection and set merges in near-constant time.

Kruskal's C++ Pseudocode

```
vector<Edge> Graph::Kruskal() {
    vector<Edge> mst;
    DisjSets ds(numVertices);
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.weight < b.weight;
    });
    int edgesCount = 0;
    for (const Edge& edge : edges) {
        int root1 = ds.find(edge.src);
        int root2 = ds.find(edge.dest);
        if (root1 != root2) {
            mst.push_back(edge);
            ds.unionSets(root1, root2);
            edgesCount++;
            if (edgesCount == numVertices - 1) break;
        }
    }
    return mst;
}
```

Kruskal's Algorithm Step-by-Step (Initial Forest)



Kruskal Status:

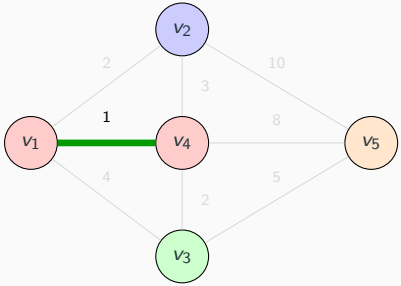
Edges sorted by weight. Forest initialized.

Current DSU Forest:

{v1}, {v2}, {v3}, {v4}, {v5}

Total MST Cost: **0**

Kruskal's Algorithm Step-by-Step (Step 1: Edge (v1, v4) w=1)



Kruskal Status:

Examining edge: **(v1, v4)** with weight **1**

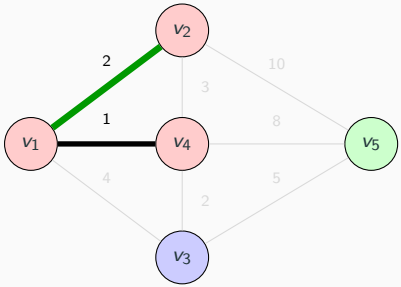
Status: **Accepted (Different Sets)**

Current DSU Forest:

{v1, v4}, {v2}, {v3}, {v5}

Total MST Cost: **1**

Kruskal's Algorithm Step-by-Step (Step 2: Edge (v1, v2) w=2)



Kruskal Status:

Examining edge: **(v1, v2)** with weight **2**

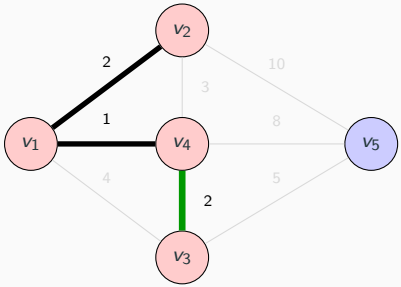
Status: **Accepted (Different Sets)**

Current DSU Forest:

{v1, v2, v4}, {v3}, {v5}

Total MST Cost: **3**

Kruskal's Algorithm Step-by-Step (Step 3: Edge (v3, v4) w=2)



Kruskal Status:

Examining edge: **(v3, v4)** with weight **2**

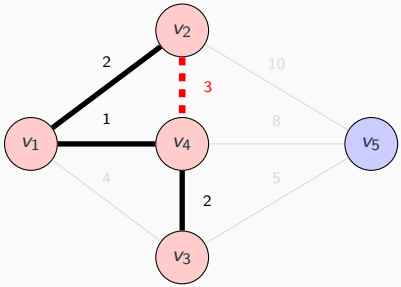
Status: **Accepted (Different Sets)**

Current DSU Forest:

{v1, v2, v3, v4}, {v5}

Total MST Cost: **5**

Kruskal's Algorithm Step-by-Step (Step 4: Edge (v2, v4) w=3)



Kruskal Status:

Examining edge: **(v2, v4)** with weight **3**

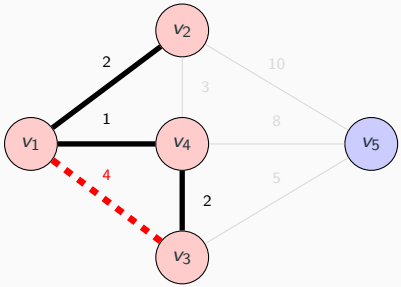
Status: **Rejected (Same Set - Cycle!)**

Current DSU Forest:

{v1, v2, v3, v4}, {v5}

Total MST Cost: **5**

Kruskal's Algorithm Step-by-Step (Step 5: Edge (v1, v3) w=4)



Kruskal Status:

Examining edge: **(v1, v3)** with weight **4**

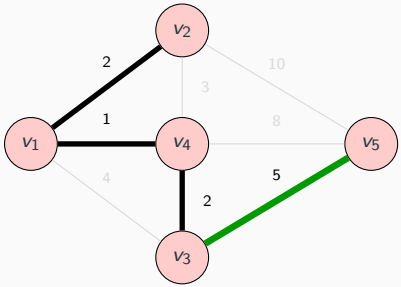
Status: **Rejected (Same Set - Cycle!)**

Current DSU Forest:

{v1, v2, v3, v4}, {v5}

Total MST Cost: **5**

Kruskal's Algorithm Step-by-Step (Step 6: Edge (v3, v5) w=5)



Kruskal Status:

Examining edge: **(v3, v5)** with weight **5**

Status: **Accepted (Different Sets)**

Current DSU Forest:

{v1, v2, v3, v4, v5}

Total MST Cost: **10**

Applications of MST

Real-World Applications of MST

- **Low-Cost Network Design**:
  - Laying out fiber-optic cables, phone lines, power grids, or pipelines.
  - Connecting N cities with minimum total cost while ensuring connectivity.
- **Approximation of NP-Hard Problems**:
  - **Metric Travelling Salesperson Problem (TSP)**: An MST can be used to construct a tour that is at most 2 times the length of the optimal tour (using DFS on the MST).
- **Clustering**:
  - Building a Single-Linkage Dendrogram in hierarchical clustering.
  - Deleting the $K-1$ longest edges in an MST splits the vertices into K clusters with maximum spacing.